Newton School of Technology

10

Design Patterns - Introduction & Creational Patterns

by Raghav Khandelwal

Join the lecture online on your dashboard.

Recap of Previous Lectures!

Design Patterns: Recipes for better code

- Design patterns are **reusable solutions to commonly occurring problems** in software design

- **Analogy:** Think of it like a **recipe book for coding**

You don't reinvent how to make biryani - you follow a proven recipe

Key Characteristics

- **Not a final design** - but a template or recipe
- **Reusable solution** to problems that occur repeatedly
- **Battle-tested** by the software development community

The Four Forces behind every pattern

- **Flexibility:** Can we adapt to new requirements?
- **Testability:** Can we write effective unit tests?
- **Coupling:** Can we change one without affecting others?
- **Efficiency:** Performance considerations

The Pattern Family: 3 categories

- **Creational Patterns**
  Focus on object **creation** mechanisms

- **Structural Patterns**
  Focus on object **composition** and **relationships**

- **Behavioral Patterns**
  Focus on communication and **interaction** between objects

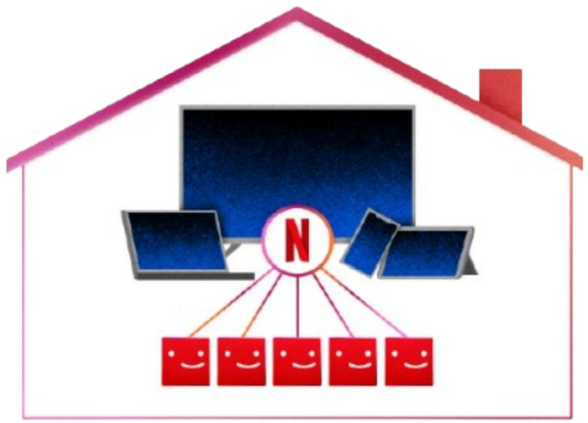
Singleton Design Pattern

“One king to Rule Them All”

What is Singleton Pattern ?

- We need **exactly one instance** of a class, accessible globally throughout the application.

- **Real-World Example: Netflix Account** - ONE account, but you can watch on any device (phone, laptop, TV)



Why Only One ?

- **Centralized Control of a Shared Resource** - Ensures that resources like configuration, logging, cache, or DB connections are managed consistently

- **Prevents Conflicts & Inconsistent State** - Multiple instances may lead to state mismatches

- **Efficient Use of Memory & Expensive Objects** - Some objects are costly to create

Building a Singleton: A Classic Recipe

Three Key Components:

- **Private Constructor -** No one can use new to create instances
- **Static Field -** Holds the single instance, shared across all usage
- **Static Access Method: getInstance() -** Provides the single instance. Creates it on first request (lazy initialization). Returns existing instance on subsequent calls

Singleton: Code Implementation

```
class Singleton {
  private static instance: Singleton;   // holds the single
instance
  private constructor() {
    console.log("Singleton instance created");
  }

  static getInstance(): Singleton {
    if (!Singleton.instance) {
      Singleton.instance = new Singleton();
    }
    return Singleton.instance;
  }

  sayHello() {
    console.log("Hello from Singleton!");
  }
}

// Usage
const s1 = Singleton.getInstance();
const s2 = Singleton.getInstance();

s1.sayHello();

console.log("Are they same?", s1 === s2);
```

When One Becomes a Problem

Singleton's Pitfalls

- **Global State and Global Dependency** - Acts like a global variable anyone can access

- **Unit Testing Nightmare -** Cannot mock or replace the Singleton in tests.
  Tests become interdependent (one test affects another)

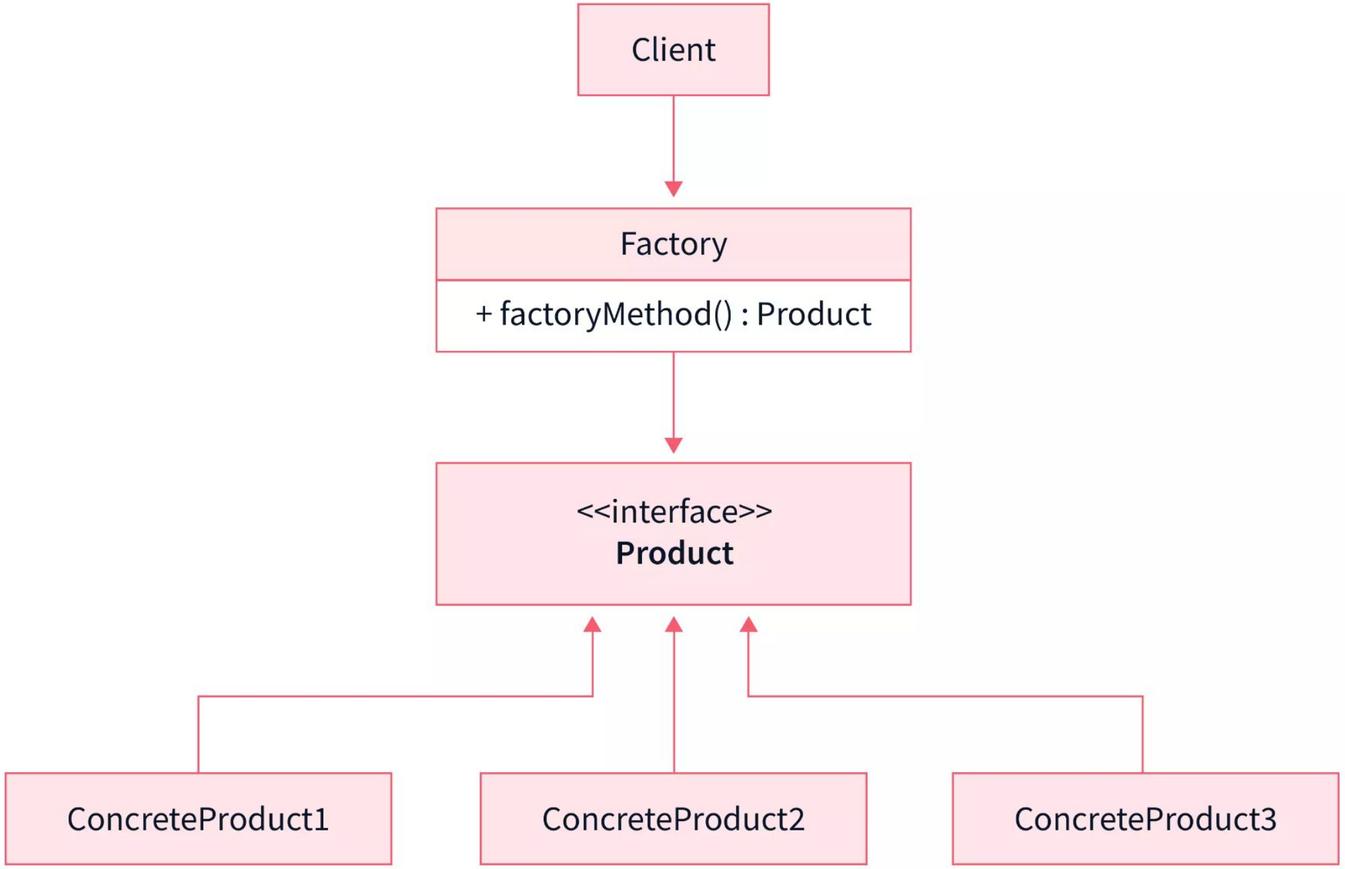
Factory Design Pattern

“The object Chef”

Simple Factory: Centralizing Object Creation

- A **single method** (often static) or class responsible for creating objects based on input parameters
- **Key Benefit:** Client code is decoupled from concrete classes
- **Structure:**
  - ➔ Abstraction/Interface
  - ➔ Concrete Products
  - ➔ Simple Factory

Simple Factory: Flowchart



Why do we need Factory Pattern ?

The Problem:

- Object creation logic becomes complex
- Need to delegate creation to subclasses
- Want to hide the actual class type from the client

Real Life Disaster Scenario

Imagine you hard-coded WhatsApp everywhere in your college app (200+ places in code)

What happens when:

❌ College bans WhatsApp (yes, some do!)

❌ Students complain: "I want email instead!"

Solution without Factory Pattern

- Search through **200+** files
- Change **new WhatsAppSender()** to **new EmailSender()** everywhere.
- Test every single place you changed
- Miss a few places → **bugs!**

Factory in Action: Example

Real-Life Scenario: Your college needs to send important alerts to students (class cancellations, exam schedules, fest updates):

Student Preferences:

- Rajesh prefers **WhatsApp**
- Priya prefers **Email**
- New students want **Telegram**

The Solution: Notification Factory

- **NotificationSender Interface -** Concrete senders inherits this class
- **Concrete Senders -** WhatsAppSender , EmailSender, TelegramSender
- **NotificationFactory -** class responsible for sending messages based on input parameters

Coding Activity

Code The Notification Factory

Benefits For Factory

- **Change in ONE place** - update config or user preference
- **Add new types easily** - Telegram, Discord, Slack (factory handles it)
- **Test different implementations** - can easily swap for testing
- **No code changes in 200+ places** - client code stays untouched

Your Turn

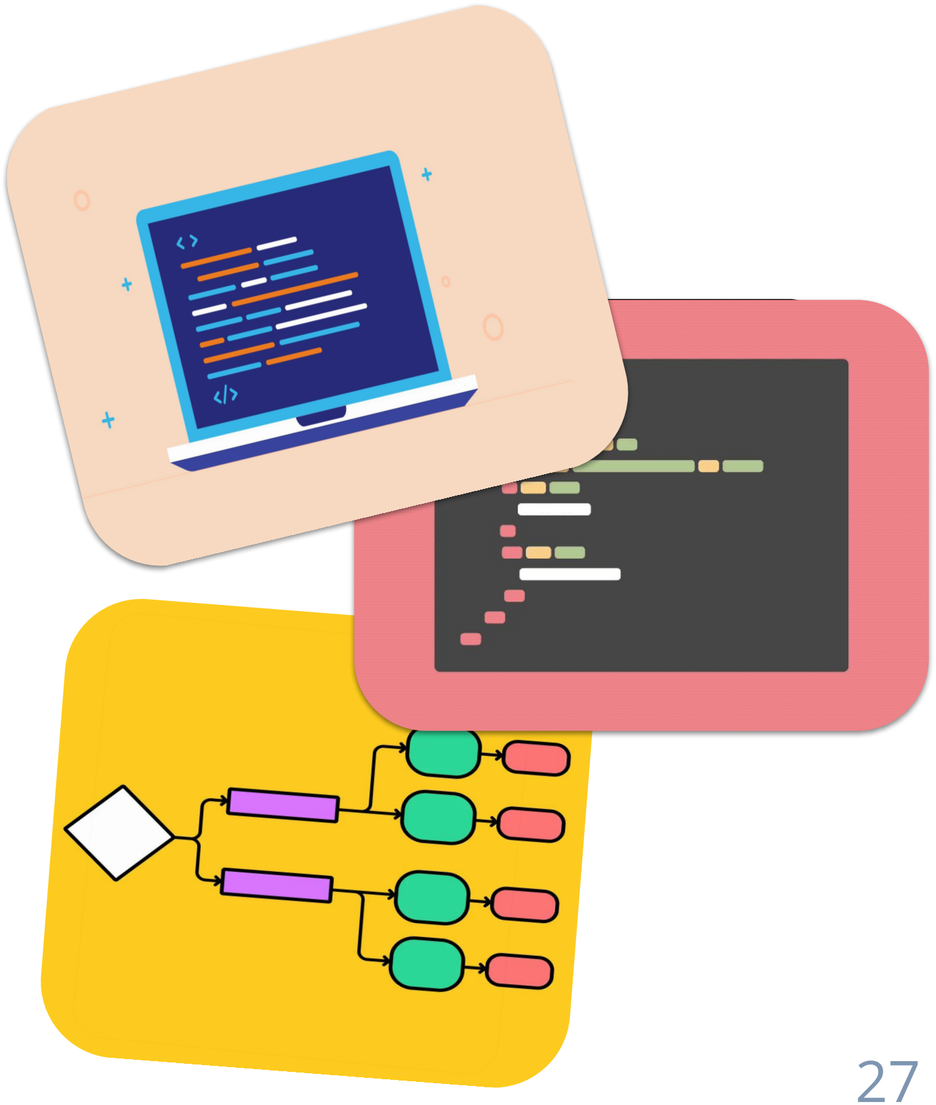
Quick Check: Test your Understanding

- **Question 1:** Name the three categories of design patterns.
- **Question 2:** What is the primary problem the Singleton pattern tries to solve?
- **Question 3:** Why is the classic Singleton considered an anti-pattern by many developers today?

MCQ Time ..

Summary

- **Design Patterns** are battle-tested solutions that provide a common language
- **Creational Patterns** manage object instantiation safely and flexibly
- **Singleton** guarantees a single instance but risks fragile global state and poor testability
- **Factory** abstracts creation logic, allowing clients to request objects by interface rather than concrete type



Newton School of Technology

Thank You ..